

Predicate locking granularity in extensible index structures

Andrey Borodin

Аннотация

Serializable snapshot isolation detects a write that would have changed the answer of a read that already happened. To do so it needs the reader to record what it read, as a predicate rather than as a set of rows, and the writer to check whether its new value falls under any recorded predicate. Where an access method offers no support for this, the predicate degenerates to the whole relation and every insertion conflicts with every scan.

We report predicate locking at page granularity for the generalized search tree, and the argument that makes page granularity correct there: the key of a page covers every entry on it, so a value that would be placed on a page necessarily falls inside the region a scan of that page examined. The same reasoning fails for the generalized inverted index, whose scans do not read a contiguous region of a key space, and we describe the two ways it failed in practice — a scan that found no match recorded nothing, although the point of a predicate lock is to lock the gap; and enabling deferred insertion while a transaction was running allowed later insertions to bypass locks already taken. The corrected principle is stated in the description of the structure rather than only in code, so that subsequent changes can be checked against it, and we argue that this is the minimum a concurrency-control decision has to leave behind if it is to survive maintenance.

1 Introduction

Serializable snapshot isolation [2] achieves serializability without read locks by detecting, at commit time, the dependency patterns that could produce a non-serializable schedule. One of the dependencies it must see is between a transaction that read a range of values and a transaction that inserted a value into that range — the phantom. The reader did not see the new row, and nothing in its read set records that it *would* have seen it, because the row did not exist.

The mechanism is a predicate lock: the reader registers the region it examined, the writer checks whether its new value falls inside any registered region. What makes this cheap or expensive is granularity. An access method that offers no support forces the region to be the entire relation, and then every insertion conflicts with every scan, which turns serializable isolation from a correctness guarantee into an availability problem.

For a B-tree the granularity question has an obvious answer, because a scan reads a contiguous interval of a total order and the interval is exactly the predicate. For an extensible index framework there is no order, the key type is not known to the core, and what a scan examined has no description outside the structure itself. This paper is about what granularity is available in that setting, and about a case where the natural answer for one structure was carried over to another where its justification did not hold.

Contributions.

1. Page-granularity predicate locking for the generalized search tree, with the covering argument that establishes its correctness (Section 2).
2. An account of two defects in the corresponding mechanism for the generalized inverted index, and the revised principle (Section 3).
3. The observation that a concurrency-control decision must be recorded in the description of the structure and not only in code, with the reason (Section 4).

2 Page granularity for the generalized search tree

A scan of a generalized search tree visits a set of pages, and on each page it evaluates consistent against every entry. What did it read? Not the entries it returned, and not the entries it examined: it read the *region* described by the keys of the pages it visited, because had a matching entry been present anywhere in that region, the scan would have found it.

That observation gives the granularity. A scan takes a predicate lock on every page it examines; an insertion checks for a lock on the page where its entry will be placed. The rule is correct because of a property the structure maintains anyway: the key of a page covers every entry on the page. Hence if a new value would be placed on a given page, it falls within that page's key, that is within a region the scan of that page examined. There is no way for an insertion to land on a page and yet be outside what a scan of that page would have covered.

Two implementation subtleties are worth recording because both are easy to get wrong in the direction of false conflicts or missed ones. The check must not fire during index construction, where entries are placed without any concurrent reader having a stake in them. And splitting a node inserts a downlink into the parent, which is an insertion into a page but not an insertion of user data; treating it as the latter produces conflicts that correspond to nothing in the user's schedule.

Implemented in 3ad55863e93, released in PostgreSQL 11.

3 Where the argument does not carry over

The inverted index stores, for each key, the list of rows containing it: an entry tree over keys, whose leaves hold either an inline posting list or the root of a posting tree. Page granularity was implemented for it by analogy. The analogy is where the trouble started, because the covering argument of Section 2 does not hold: a page of the entry tree is not a region of a space that an insertion must fall into, and a scan of this structure does not examine a contiguous region at all.

Two defects followed, and they have opposite shapes.

A scan that finds nothing must still lock something. A scan that found no matching entry recorded no predicate lock. This is the natural reading of “lock what you read” and it is wrong, because the purpose of a predicate lock is to lock the *gap*. A query asking for a key that does not exist has read exactly the fact that it does not exist, and an insertion creating it must conflict. Recording nothing means the phantom is invisible.

Turning on deferred insertion mid-flight bypasses existing locks. The index supports a pending list into which insertions may be deposited and applied later. If that mode is enabled while transactions are running, subsequent insertions go to the pending list and no longer conflict with locks that were already taken on entry tree pages — the locks describe a structure the insertion no longer touches. A configuration change thus silently weakens isolation for transactions already in progress.

The revised principle. A scan always locks the metapage, and this lock represents the reading of the pending list whether or not one currently exists — which is what makes the mode change harmless. And a scan always locks the entry tree page it consulted, including when it found no match, which is what makes gaps visible. In the other direction, locking was found to be excessive and was reduced: every element of a posting tree belongs to a single key value, so locking the root of that tree is sufficient and locking its pages individually gains nothing.

Implemented in 0bef1c0678d, released in PostgreSQL 11. A cost of the mechanism showed up later in an unexpected place: the additional locking slowed the isolation test suite enough under instrumentation to be worth reporting [1], which is a reminder that correctness machinery is paid for on every scan, not only on the schedules it exists to catch.

4 Writing the principle down where it can be checked

The revision above was recorded in the prose description of the structure that accompanies the source, not only in the code implementing it. We consider this part of the result rather than a matter of documentation hygiene, for a reason specific to concurrency control.

A rule such as “a scan always locks the metapage, and this represents reading the pending list whether or not one exists” is not visible in the code that implements it. What the code shows is a lock being taken at a particular line; why it is taken there, and what would break if it were taken elsewhere or made conditional, is nowhere in the text. A later change that makes the lock conditional — for instance because profiling shows the pending list is usually absent — looks locally like a valid optimisation, passes every functional test, and reintroduces exactly the defect described above. The decision has to be written where the person making that later change will encounter it.

This generalizes past predicate locking, and it is the same argument we have made elsewhere about verifying structural invariants: properties that no functional test observes must be recorded in a form that can be checked by something other than running the code. An invariant checker is one such form. A stated principle in the description of the structure is a weaker one, since it is checked by a human reader rather than by a program, but it is much cheaper and it is available for properties that no program can practically check.

5 Evaluation

6 Limitations

Page granularity is coarser than the predicate a query actually describes, and on a page holding entries spread over a wide region it will report conflicts that a finer mechanism would not. We have not characterized how often that happens on real workloads; the argument for page granularity is that it is sound and cheaply computable from what the structure already maintains, not that it is tight.

The correctness argument of Section 2 depends on the covering property of keys, and therefore on the operator class maintaining it. An operator class whose union does not produce a covering key would break predicate locking along with search itself, so this is not an additional assumption; but it is worth naming, since it is the point at which the framework trusts code it did not write.

Section 3 reports defects found in production and by review rather than a systematic search. We do not claim the revised principle is complete, and the history of this particular mechanism does not encourage the claim.

7 Conclusion

Predicate locking is the point where an access method has to describe what a query read, rather than what it returned, and the two differ exactly by the rows that do not exist. For a structure with a covering invariant the description is available for free: the key of a page is a sound over-approximation of what a scan of that page examined, so page granularity is both correct and cheap.

The instructive part is the failure of the analogy. The same granularity was applied to a structure without that invariant, and produced a mechanism that looked equivalent and was not: it recorded nothing when a scan found nothing, and it could be bypassed by a configuration change. Neither defect is visible in a test that runs a workload and checks its answers, because both concern schedules that the test does not generate. What they have in common with the rest of our work on this framework is the form of the eventual remedy: state the property, in a place where a later change will be checked against it, rather than trusting that the code will keep saying what it currently says.

Acknowledgements

Список литературы

- [1] Andres Freund et al. GIN predicate locking slows down valgrind isolationtester runs. Discussion on the pgsql-hackers mailing list, 2018.
- [2] Dan R K Ports and Kevin Grittner. Serializable snapshot isolation in postgresql. In *Proceedings of the VLDB Endowment*, volume 5, pages 1850–1861, 2012.